

ЄФВВ ІТ 2026: посібник, синхронізований із v27 тестами

Не збірник старих питань, а карта понять для складання: 10 блоків програми, 140-питальні варіанти, схеми, пастки, приклади й мінімум трасування коду без перевантаження.



Algorithms and Data Structures



Computer Architecture and Data Representation



Databases, SQL and Relational Algebra



Software Engineering, UML and Design Patterns



Cybersecurity and Cryptography



Math for IT: Logic, Derivatives, Probability, Sets



Networks, OSI, Subnets and Protocols



Operating Systems



Programming Paradigms and Code Tracing



AI, ML and Data Science

Як читати: спочатку схема, потім пастки, потім 6 прикладів того, що питає v27-банк у цьому блоці. Після кожного блоку варто пройти 10-15 питань у practice або цілий variant у simulation.

CONTENTS

Маршрут читання

Pages	Block	What it covers
3-4	 Algorithms and Data Structures	Big-O, sorting, search, graphs, trees, stacks/queues, classic hard tasks.
4-5	 Computer Architecture and Data Representation	CPU, cache, RAM/ROM, compiler/interpreter/linker/JIT, binary, signed/unsigned, IEEE 754.
5-6	 Databases, SQL and Relational Algebra	Relational model, keys, normalization, ER, SQL clauses, indexes, GRANT, ACID, OLTP/OLAP.
6-7	 Software Engineering, UML and Design Patterns	SDLC, Agile/Scrum, UML diagrams, testing, cohesion/coupling, patterns.
7-8	 Cybersecurity and Cryptography	CIA, DAD, DDoS, hashes, Ukrainian crypto standards, access control, signatures.
8-9	 Math for IT: Logic, Derivatives, Probability, Sets	Derivative rules, stationary points, probability, combinatorics, set operations, implication.
9-10	 Networks, OSI, Subnets and Protocols	OSI levels, frames/packets/segments, switch/router, ARP/DNS/DHCP, TCP/UDP, subnet masks.
10-11	 Operating Systems	Processes, threads, scheduling, deadlock, mutex/semaphore, virtual memory, paging, kernel/user mode.
11-12	 Programming Paradigms and Code Tracing	Procedural, OOP, functional, encapsulation, inheritance, polymorphism, interfaces, recursion, one lightweight tracing drill.
12-13	 AI, ML and Data Science	kNN, classification/regression/clustering, SVM, neural networks, activation functions, overfitting, IoU.

Баланс тесту

IT v27 має 10 блоків по 14 питань у кожному варіанті. Це зроблено, щоб тренувати програму 2026, а не повторювати один старий тест.

Variant	Questions	Structure
IT v1	140	10 blocks × 14 questions
IT v2	140	same concepts, different formulations
IT v3	140	same concepts, different traps

Про задачі з а, b, x, y, i: вони потрібні лише як перевірка уважності до коду. У тренувальному IT-варіанті достатньо 1-2 таких змішаних задач; основний бал

дає впізнавання понять, моделей, алгоритмів, мереж, БД, ООП, ML і безпеки.



Algorithms and Data Structures

Focus: Big-O, sorting, search, graphs, trees, stacks/queues, classic hard tasks.

Schema

ALGORITHMS

```
|
|-- complexity: O(1), O(log n), O(n), O(n log n), O(n^2)
|-- sorting: bubble, selection, insertion, merge, quick, heap
|-- structures: stack, queue, linked list, BST, AVL, heap
|-- graphs: BFS, DFS, shortest path, planar graph
|-- task types: search, sort, optimization, dynamic programming
```

Scientific explanation

An algorithm is a finite procedure for solving a task. Complexity estimates how the number of operations grows when input size n grows. Tests often ask not for exact time in seconds, but for the growth class.

Life analogy

Sorting papers: bubble swaps neighbours, selection chooses the smallest pile item, insertion places a new card into a sorted hand, merge splits piles and merges them, quick chooses a pivot and sends papers left/right.

Typical traps

- Binary search needs sorted data.
- Quick sort is often $O(n \log n)$, but worst case can be $O(n^2)$.
- Merge sort has guaranteed $O(n \log n)$, but needs extra $O(n)$ memory.
- Stack is LIFO, queue is FIFO.
- AVL is not just any BST; it is height-balanced.

Mini-drill

- If the prompt says pivot/partition -> Quick Sort.
- If it says merge sorted halves -> Merge Sort.
- If it says repeatedly find minimum -> Selection Sort.
- If it says insert into sorted-left -> Insertion Sort.

What v27 tests ask here

- **big o:** Що в записі складності $O(n)$ позначає літера n ?
- **sorting:** Який алгоритм сортування має гарантовану складність $O(n \log n)$ у найгіршому випадку?
- **quick sort:** У Quick Sort обрано опорний елемент. Що відбувається далі з масивом?
- **binary search:** Яка умова є обов'язковою для коректної роботи бінарного пошуку?
- **avl tree:** Яке дерево пошуку підтримує різницю висот лівого і правого піддерев кожного вузла не більше ніж 1?
- **stack queue:** Яка структура даних працює за принципом LIFO?

Sorting quick table

Algorithm	Best	Average	Worst	Stable?	Memory	Exam clue
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	yes	$O(1)$	neighbour swaps
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	usually no	$O(1)$	find minimum
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	yes	$O(1)$	insert into sorted-left
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	yes	$O(n)$	divide + merge
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	usually no	$O(\log n)$	pivot + partition
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	no	$O(1)$	binary heap
Counting	$O(n+k)$	$O(n+k)$	$O(n+k)$	can be yes	$O(k)$	counters for values
Radix	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	yes, with stable pass	$O(n+k)$	digit by digit



Computer Architecture and Data Representation

Focus: CPU, cache, RAM/ROM, compiler/interpreter/linker/JIT, binary, signed/unsigned, IEEE 754.

Schema

COMPUTER DATA

```
|  
|-- bit: 0 or 1  
|-- byte: 8 bits  
|-- unsigned: all combinations are non-negative  
|-- signed: first bit helps encode negative values  
|-- floating point: sign + exponent + mantissa
```

Scientific explanation

The same bits can mean different values depending on the type. 11111111 can be 255 as unsigned 8-bit, but -1 as signed two's complement. The CPU follows the instruction and type context; it does not guess meaning by itself.

Life analogy

The same text '01.02' can mean February 1 in a date or 1 minute 2 seconds in sport. Bits are the same; context gives meaning.

Typical traps

- 10000000 in 8 bits is 128 unsigned, but -128 signed.
- 256 needs 9 bits: 1 0000 0000.
- IEEE 754 exponent stores biased exponent, not the raw order.
- Compiler, interpreter, linker and JIT are different stages/tools.

Mini-drill

- $1011.01_2 = 1.01101_2 * 2^3$, so raw order is 3.
- For IEEE single precision exponent field: order + 127.
- To get -10 in 8-bit two's complement: 00001010 -> invert 11110101 -> +1 = 11110110.

What v27 tests ask here

- **harvard**: Головна ознака Гарвардської архітектури:
- **von neumann**: У фон-нейманівській архітектурі програма і дані зазвичай:
- **cache**: Для чого потрібна cache-пам'ять процесора?
- **cpu**: Який компонент CPU виконує арифметичні та логічні операції?
- **compiler interpreter**: Яке твердження про компілятор та інтерпретатор є правильним?
- **jit**: Що таке JIT-компіляція?



Databases, SQL and Relational Algebra

Focus: Relational model, keys, normalization, ER, SQL clauses, indexes, GRANT, ACID, OLTP/OLAP.

Schema

DATABASE THINKING

```

|
|-- conceptual model: business entities and relations
|-- logical model: tables, keys, normalization
|-- physical model: indexes, storage, files
|-- SQL: SELECT/FROM/WHERE/GROUP BY/HAVING/ORDER BY
|-- relational algebra: selection, projection, join, difference, union

```

Scientific explanation

Relational databases model data as relations. A row is a tuple, a column is an attribute, and atomicity means one cell should contain one indivisible value. Relational algebra is the formal language behind SQL operations.

Life analogy

A library catalogue: primary key is the unique inventory number, foreign key points to another card, index is a fast lookup shelf, normalization prevents writing the same author details in every place.

Typical traps

- WHERE filters rows before grouping; HAVING filters groups after GROUP BY.
- Index usually helps on columns used in WHERE/JOIN, not randomly on every SELECT column.
- Only in one table means symmetric difference: $(A-B) \cup (B-A)$.
- Projection π keeps columns; selection σ filters rows.

Mini-drill

- Names of cats only in one table:
 $\pi_{\text{name}}((\text{CatsA} - \text{CatsB}) \cup (\text{CatsB} - \text{CatsA}))$.
- WHERE email LIKE 'Alex%' -> index email_address.
- GRANT gives permissions.

What v27 tests ask here

- **relational model**: Яка вимога характерна для відношення реляційної моделі?
- **keys**: Primary key у таблиці потрібен, щоб:
- **foreign key**: Foreign key зазвичай:
- **normalization**: Для чого потрібна нормалізація БД?
- **models**: Яка модель даних передбачає нормалізацію таблиць?
- **er model**: ER-модель насамперед показує:

Software Engineering, UML and Design Patterns

Focus: SDLC, Agile/Scrum, UML diagrams, testing, cohesion/coupling, patterns.

Schema

```
SOFTWARE ENGINEERING
|
|-- lifecycle: waterfall, iterative, agile
|-- UML structure: class diagram
|-- UML behaviour: sequence/use case diagrams
|-- quality: testing, cohesion, coupling
|-- patterns: creational, structural, behavioural
```

Scientific explanation

Software engineering organizes development so the system can be specified, built, tested, changed and maintained. UML is a modelling language: each diagram answers a different question about the system.

Life analogy

Class diagram is like a building plan; sequence diagram is like a video of people passing documents; use case diagram is like a list of who uses the service and why.

Typical traps

- Class diagram shows static structure; sequence diagram shows message order over time.
- Bridge is structural. Singleton, Builder, Prototype are creational.
- Unit tests check small units; integration tests check interaction.
- Waterfall is sequential; Agile is iterative.

Mini-drill

- Attributes/methods/classes -> UML class diagram.
- Messages over time -> UML sequence diagram.
- Actors and goals -> UML use case diagram.

What v27 tests ask here

- **waterfall**: Що передбачає водоспадна модель життєвого циклу?
- **agile**: Agile-підхід зазвичай робить акцент на:
- **scrum**: У Scrum короткий фіксований цикл роботи називають:
- **uml class**: Яка UML-діаграма показує класи, атрибути, методи та зв'язки між класами?
- **uml sequence**: Яка UML-діаграма показує порядок повідомлень між об'єктами в часі?
- **uml use case**: Use case diagram насамперед показує:



Cybersecurity and Cryptography

Focus: CIA, DAD, DDoS, hashes, Ukrainian crypto standards, access control, signatures.

Schema

```
SECURITY
|
|-- CIA: confidentiality, integrity, availability
|-- attacker model: disclosure, alteration, denial
|-- crypto: hash, symmetric, asymmetric, signature
|-- access: MAC, DAC, RBAC
|-- attacks: DDoS, ransomware, phishing
```

Scientific explanation

Information security protects data and services. Confidentiality hides information, integrity prevents unauthorized changes, availability keeps systems usable. Cryptographic hash functions create fixed-size digests and are not reversible encryption.

Life analogy

CIA is a guarded archive: only allowed people read it, pages are not changed, and the archive is open when needed. DDoS attacks the third part: availability.

Typical traps

- DDoS attacks availability, not integrity.
- Kupyna is a hash function; Kalyna is a block cipher.
- Hashing is not encryption because it is not meant to be reversed.
- MAC in access control is Mandatory Access Control, not MAC address.

Mini-drill

- CIA opposite in attacker language: DAD = Disclosure, Alteration, Denial.
- Digital signature checks author and integrity.
- RBAC gives permissions through roles.

What v27 tests ask here

- **cia**: Яка тріада є базовою моделлю захисту інформації?
- **dad**: Яку модель дій зловмисника часто протиставляють CIA?
- **ddos**: DDoS-атаки насамперед порушують:
- **hash**: Криптографічна геш-функція призначена для:
- **kupyna**: Який український стандарт є криптографічною геш-функцією?
- **symmetric**: Симетричне шифрування означає, що:



Math for IT: Logic, Derivatives, Probability, Sets

Focus: Derivative rules, stationary points, probability, combinatorics, set operations, implication.

Schema

```
MATH IN IT TESTS
|
|-- derivative: rate of change, optimization, ML gradients
|-- probability: favourable / total
|-- combinatorics: OR=sum, AND=product, roles=arrangements, no roles=combinations
|-- sets: union, intersection, difference, symmetric difference
|-- logic: implication is false only when true -> false
```

Scientific explanation

Mathematical questions in IT tests usually check the tool, not deep theory. Derivatives support optimization and learning; probability checks risk/counting; set operations mirror database and logic operations.

Life analogy

Derivative is a speedometer for a function. Probability is how many good tickets are in the box. Combinatorics is counting routes without listing all routes by hand.

Typical traps

- OR between disjoint choices -> add; AND sequence -> multiply.
- Committee/team without roles -> combinations.
- Chair/deputy/secretary -> arrangements because roles matter.
- $P \rightarrow Q$ is false only when P is true and Q is false.

Mini-drill

- 20 details, 4 defective -> standard probability is 16/20.
- $\frac{d}{dx} x^3 = 3x^2$.
- $\frac{d}{dx} (1 - e^{-ax}) = a \cdot e^{-ax}$.

What v27 tests ask here

- **derivative:** Обчисліть границю $\lim_{\Delta x \rightarrow 0} ((x+\Delta x)^3 - x^3)/\Delta x$.
- **chain rule:** Визначте похідну $y(x)=1-e^{-a \cdot x}$, якщо $a=\text{const}$.
- **stationary points:** Стаціонарна точка функції кількох змінних — це точка, де:
- **probability:** У партії з 20 деталей 4 браковані. Яка ймовірність навмання взяти стандартну деталь?
- **sum rule:** Об'єкт а можна вибрати 6 способами, а об'єкт b — іншими 4 способами. Треба обрати будь-який один: а або b. Скільки способів?
- **product rule:** 6 варіантів першої дії і 4 варіантів другої. Треба виконати обидві дії. Скільки способів?



Networks, OSI, Subnets and Protocols

Focus: OSI levels, frames/packets/segments, switch/router, ARP/DNS/DHCP, TCP/UDP, subnet masks.

Schema

```
NETWORK STACK
|
|-- L2: frame, MAC, switch
|-- L3: packet, IP, router
|-- L4: TCP/UDP, ports
|-- L7: HTTP, DNS, SMTP, SSH
|-- subnet: network bits + host bits
```

Scientific explanation

OSI separates network communication into layers. Each layer has its own unit and responsibility. Subnetting splits an IP network into smaller address ranges, but network and broadcast addresses reduce usable hosts.

Life analogy

Sending a parcel: MAC is local apartment delivery, IP is city-to-city routing, port is the office room, HTTP/DNS are the application-level services.

Typical traps

- Switch -> MAC/frame/L2; router -> IP/packet/L3.
- DNS resolves names; DHCP gives network settings.
- 64 required hosts need more than 64 total addresses, because network and broadcast are reserved.
- HTTP 404 means resource not found.

Mini-drill

- /26 gives 64 total addresses and 62 usable hosts.
- Need 64 usable hosts -> /25, mask 255.255.255.128.
- ATM backbone clue -> cell switching.

What v27 tests ask here

- **osi**: На якому рівні OSI працює IP?
- **pdu**: Що правильніше зіставляє PDU з рівнями?
- **switch**: Комутатор у LAN зазвичай пересилає кадри на основі:
- **router**: Маршрутизатор насамперед потрібен для:
- **arp**: ARP у локальній мережі використовується для:
- **tcp udp**: TCP відрізняється від UDP тим, що TCP:

Operating Systems

Focus: Processes, threads, scheduling, deadlock, mutex/semaphore, virtual memory, paging, kernel/user mode.

Schema

```
OPERATING SYSTEM
|
|-- process: program instance with address space
|-- thread: execution path inside a process
|-- scheduler: decides who gets CPU
|-- memory: virtual memory, paging
|-- sync: mutex, semaphore, race condition, deadlock
```

Scientific explanation

An operating system manages hardware resources and gives programs safe abstractions: processes, files, memory, devices and scheduling. Many test questions ask what component is responsible for a specific job.

Life analogy

OS is a building manager: gives rooms, schedules elevators, controls keys, handles device requests and prevents tenants from damaging each other.

Typical traps

- Process has its own address space; threads share process memory.
- Mutex is one-at-a-time access; semaphore can allow limited multiple access.
- Deadlock is circular waiting.
- Kernel mode has privileged access; user mode is restricted.

Mini-drill

- Race condition -> result depends on timing/order of threads.
- Driver -> lets OS control hardware.
- Buffer -> smooths speed differences in I/O.

What v27 tests ask here

- **process thread**: Процес відрізняється від потоку тим, що процес:
- **scheduler**: Планувальник ОС вирішує:
- **deadlock**: Deadlock — це ситуація, коли:
- **mutex**: Mutex потрібен для:
- **semaphore**: Semaphore відрізняється від простого mutex тим, що може:
- **virtual memory**: Віртуальна пам'ять дає змогу:



Programming Paradigms and Code Tracing

Focus: Procedural, OOP, functional, encapsulation, inheritance, polymorphism, interfaces, recursion, one lightweight tracing drill.

Schema

PROGRAMMING PARADIGMS

```
|
|-- procedural: functions/procedures + data
|-- OOP: class, object, state, methods
|-- functional: pure functions, less mutable state
|-- tracing: 1-2 mixed tasks max, update variables step by step
|-- recursion: function calls itself with a smaller case
```

Scientific explanation

Programming paradigms are ways to organize code and thinking. Procedural code emphasizes steps, OOP emphasizes objects with state and behaviour, functional programming emphasizes transformations through functions.

Life analogy

Procedural is a recipe; OOP is a set of smart devices with their own buttons and state; functional is a clean pipeline where each step transforms input into output.

Typical traps

- Class is a template; object is a concrete instance.
- Encapsulation is controlled access to state, not just the word private.
- Inheritance is is-a; composition is has-a.
- Tracing variables must be done line by line, not by guessing.

Mini-drill

- Make a table: iteration | a | b | x | y | result.
- Keep tracing to 1-2 mixed tasks: enough to learn the table method, not enough to replace theory.
- If assignment changes x, the next line uses the new x.

What v27 tests ask here

- **procedural**: Що покладено в основу процедурного програмування?
- **oop**: Що є основою об'єктно-орієнтованого програмування?
- **functional**: Функціональне програмування найбільше підкреслює:
- **encapsulation**: Інкапсуляція означає:
- **inheritance**: Наслідування в ООП — це:
- **polymorphism**: Поліморфізм проявляється, коли:

Code tracing: one mixed task is enough

```
x = 2
y = 3
z = 1

for i = 1 to 3:
    z = z + x
    x = x + y
    y = y * 2
```

i	x	y	z
0	2	3	1
1	5	6	3
2	11	12	8
3	23	24	19

Rule: every next line uses the newest value. Do not mentally jump. Draw the table.



AI, ML and Data Science

Focus: kNN, classification/regression/clustering, SVM, neural networks, activation functions, overfitting, IoU.

Schema

```
MACHINE LEARNING
|
|-- supervised: labelled data, classification/regression
|-- unsupervised: clustering, dimensionality reduction
|-- kNN: class from nearest labelled neighbours
|-- SVM: separating hyperplane with margin
|-- neural net: weights + activation + training
```

Scientific explanation

Machine learning builds models from data. Training changes parameters, such as weights, so the model makes fewer errors. Evaluation checks whether the learned pattern generalizes beyond training examples.

Life analogy

kNN is asking the nearest neighbours what label they have. SVM is drawing the widest separating road between two groups. A neural network is a chain of adjustable knobs.

Typical traps

- kNN with known labels -> supervised learning.
- Classification predicts category; regression predicts number.
- Sigmoid maps to 0..1; RBF Gaussian looks like $e^{(-c \cdot x^2)}$.
- IoU is for segmentation overlap, not image quality in general.

Mini-drill

- Training NN -> changing weights.
- Overfitting -> good train score, worse test score.
- Recall -> how many true positives were found.

What v27 tests ask here

- **knn**: До якого типу навчання належить kNN, якщо відомі правильні класи навчальних об'єктів?
- **classification**: Класифікація в ML — це задача:
- **regression**: Регресія в ML — це задача:
- **clustering**: Кластеризація — це:
- **svm**: Для чого SVM використовує гіперплощину?
- **nn training**: Навчання нейронної мережі — це:

Короткий міст до інших двох тестів

TZNK

Тримати три питання: OR чи AND;
порядок важливий чи ні; чи є
заборонені варіанти, які треба відняти.

```
OR -> add  
AND -> multiply  
roles -> arrangements  
no roles -> combinations  
bad cases -> total - forbidden
```

GB English

Фокус: grammar patterns, reading
markers, conditionals, tense logic,
connectors. У reading не перекладати
все, а ловити reference words, contrast,
cause/effect.

```
however -> contrast  
therefore -> result  
unless -> if not  
although -> concession
```